

Sliding Puzzle Project

Conversation Record — Tasks 1 through 4

This document consolidates the conversation and implementation notes available in the chat context. The Task 3 and Task 4 prompts and responses are included in detail. Earlier Task 1 and Task 2 history is summarized where the original full wording was not present in the retained context.

Project files: `main.py`, `game.py`, `puzzle.py`. Controls: W/A/S/D move the tile into the blank; Q quits.

Task 1 — Ensure every generated puzzle is solvable

User request and goal

The initial task was to fix the solvability defect in `puzzle.py`. The old version randomly shuffled all tiles, which can produce an unreachable board. The required fix was to generate puzzles through legal moves while preserving the existing puzzle methods.

Implementation

The puzzle starts in solved order and is scrambled by repeatedly moving the blank into a legal neighboring position. This guarantees that every generated board remains reachable from the solved state. Immediate reversals are avoided where possible.

Updated puzzle.py after Task 1

```

import random

class Puzzle:
    def __init__(self, size=4):
        self.size = size
        self.board = self.make_board()

    def make_board(self):
        # Start from the solved arrangement so every generated board is solvable.
        tiles = list(range(1, self.size * self.size)) + [0]
        board = [
            tiles[r * self.size:(r + 1) * self.size]
            for r in range(self.size)
        ]

        # Scramble by making legal moves of the blank (0), rather than shuffling
        # tiles arbitrarily. Every move therefore keeps the board reachable.
        blank_r, blank_c = self.size - 1, self.size - 1
        previous_blank = None
        scramble_moves = max(100, self.size * self.size * 20)

        for _ in range(scramble_moves):
            neighbors = []
            for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
                nr, nc = blank_r + dr, blank_c + dc
                if 0 <= nr < self.size and 0 <= nc < self.size:
                    if (nr, nc) != previous_blank:
                        neighbors.append((nr, nc))

            if not neighbors and previous_blank is not None:
                neighbors = [previous_blank]
            if not neighbors:
                # For a 1x1 board, the blank has no legal moves.
                break

            next_r, next_c = random.choice(neighbors)
            board[blank_r][blank_c], board[next_r][next_c] = (
                board[next_r][next_c],
                board[blank_r][blank_c],
            )
            previous_blank = (blank_r, blank_c)
            blank_r, blank_c = next_r, next_c

        return board

    def blank_pos(self):
        for r in range(self.size):
            for c in range(self.size):
                if self.board[r][c] == 0:
                    return r, c

    def move(self, direction):
        r, c = self.blank_pos()
        dr, dc = {"w": (-1, 0), "s": (1, 0), "a": (0, -1), "d": (0, 1)}[direction]
        nr, nc = r + dr, c + dc
        if not (0 <= nr < self.size and 0 <= nc < self.size):
            return False
        self.board[r][c], self.board[nr][nc] = self.board[nr][nc], self.board[r][c]
        return True

    def solved(self):
        return sum(self.board, []) == list(range(1, self.size * self.size)) + [0]

```

Verification recorded

The prior assistant reported testing 1,600 generated boards for sizes 1 through 8, confirming valid tile contents and exactly one blank in each board. The tests passed.

Task 2 — Victory detection and stopping after a win

User request and goal

The next task asked to inspect `puzzle.py`, `game.py`, and `main.py`; detect the solved state; show a victory message; stop immediately after the winning move; prevent further gameplay moves after winning; show the victory message only once; preserve existing controls and Task 1's solvability fix; avoid implementing board-size selection yet; and provide complete affected code and tests.

Updated game.py after Task 2

```
import time
from puzzle import Puzzle

class SlidingPuzzle:
    def __init__(self):
        self.size = 4
        self.puzzle = Puzzle(self.size)
        self.moves = 0
        self.started = time.monotonic()
        self.won = False

    def display(self):
        print()
        for row in self.puzzle.board:
            print(" ".join(f"{x or ' ':>2}" for x in row))
        print(
            "Moves:", self.moves,
            " Time:", int(time.monotonic() - self.started), "s"
        )

    def run(self):
        # Do not restart a game that has already been won or repeat its message.
        if self.won:
            return

        print("Sliding Puzzle – W/A/S/D moves the tile into the blank. Q quits.")

        # Also handle a board that is already solved before the first input.
        if self.puzzle.solved():
            self.display()
            self.won = True
            print("Congratulations! You solved the puzzle!")
            return

        while True:
            self.display()
            key = input("> ").strip().lower()

            if key == "q":
                return
            if key not in "wasd":
                print("Use W/A/S/D.")
                continue

            if self.puzzle.move(key):
                self.moves += 1

                # Check immediately after a successful move so the game ends
                # without asking for another key after the winning move.
                if self.puzzle.solved():
                    self.display()
                    self.won = True
                    print("Congratulations! You solved the puzzle!")
                    return
            else:
                print("That move is not possible.")
```

Task 2 test summary

- A board one move away from solved was prepared and the winning key was supplied.
- The puzzle became solved, the move count became 1, and the victory message appeared exactly once.

- Calling `run()` again produced no output because the game was already won.
- The prior assistant reported that these checks passed.

main.py at that stage

```
from game import SlidingPuzzle

if __name__ == "__main__":
    SlidingPuzzle().run()
```

Task 3 — Multiple board sizes, move counting, and timing

User prompt

“Now implement Task 3: Add support for multiple puzzle sizes and ensure move counting and timing work correctly.

- Support 3×3, 4×4, and 5×5 puzzle boards.
- Allow the player to select a board size before starting a game.
- Ensure puzzle initialization, tile movement, and solved-state detection work for all three sizes.
- Preserve the solvability fix from Task 1.
- Display the number of successful tile movements.
- Increment the move counter only when a tile actually moves.
- Track and display elapsed game time.
- Reset the move counter and timer when a new game starts.
- Stop updating the timer after the puzzle is solved.
- Preserve Task 2’s victory detection and prevent moves after winning.
- Keep the existing keyboard controls and avoid unrelated changes.
- Avoid unnecessary external dependencies.
- Provide complete updated code for affected files and explain how to test each board size.
- Implement only Task 3. Do not implement Task 4 yet.

Task 3 response summary

The implementation added board-size selection in `main.py`, allowed `SlidingPuzzle` to receive a size, tracked a frozen finish time, and retained legal-move scrambling in `puzzle.py`. The response provided complete code for all three files and instructions for testing each board size.

main.py after Task 3

```
from game import SlidingPuzzle

def choose_size():
    """Ask for a supported board size before starting the game."""
    while True:
        choice = input("Choose puzzle size (3, 4, or 5): ").strip()
        if choice in {"3", "4", "5"}:
            return int(choice)
        print("Please enter 3, 4, or 5.")

if __name__ == "__main__":
    size = choose_size()
    SlidingPuzzle(size).run()
```

game.py after Task 3

```

import time
from puzzle import Puzzle

class SlidingPuzzle:
    def __init__(self, size=4):
        if size not in (3, 4, 5):
            raise ValueError("Puzzle size must be 3, 4, or 5.")

        self.size = size
        self.puzzle = Puzzle(self.size)
        self.moves = 0
        self.started = time.monotonic()
        self.won = False
        self.finished_at = None

    def elapsed_time(self):
        """Return elapsed seconds, frozen when the game is won."""
        end_time = (
            self.finished_at
            if self.finished_at is not None
            else time.monotonic()
        )
        return int(end_time - self.started)

    def display(self):
        print()
        for row in self.puzzle.board:
            print(" ".join(f"{x or ' ':>2}" for x in row))
        print("Moves:", self.moves, " Time:", self.elapsed_time(), "s")

    def _finish(self):
        if not self.won:
            self.won = True
            self.finished_at = time.monotonic()
            self.display()
            print("Congratulations! You solved the puzzle!")

    def run(self):
        if self.won:
            return

        print(
            f"Sliding Puzzle ({self.size}x{self.size}) - "
            "W/A/S/D moves the tile into the blank. Q quits."
        )

        if self.puzzle.solved():
            self._finish()
            return

        while not self.won:
            self.display()
            key = input("> ").strip().lower()

            if key == "q":
                return

            if key not in {"w", "a", "s", "d"}:
                print("Use W/A/S/D.")
                continue

            if self.puzzle.move(key):
                self.moves += 1
                if self.puzzle.solved():

```

```
        self._finish()
        return
    else:
        print("That move is not possible.")
```

puzzle.py after Task 3

```

import random

class Puzzle:
    def __init__(self, size=4):
        self.size = size
        self.board = self.make_board()

    def make_board(self):
        # Start from the solved arrangement so every generated board is solvable.
        tiles = list(range(1, self.size * self.size)) + [0]
        board = [
            tiles[r * self.size:(r + 1) * self.size]
            for r in range(self.size)
        ]

        # Scramble by making legal moves of the blank (0), rather than shuffling
        # tiles arbitrarily. Every move therefore keeps the board reachable.
        blank_r, blank_c = self.size - 1, self.size - 1
        previous_blank = None
        scramble_moves = max(100, self.size * self.size * 20)

        for _ in range(scramble_moves):
            neighbors = []
            for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
                nr, nc = blank_r + dr, blank_c + dc
                if 0 <= nr < self.size and 0 <= nc < self.size:
                    if (nr, nc) != previous_blank:
                        neighbors.append((nr, nc))

            if not neighbors and previous_blank is not None:
                neighbors = [previous_blank]
            if not neighbors:
                break

            next_r, next_c = random.choice(neighbors)
            board[blank_r][blank_c], board[next_r][next_c] = (
                board[next_r][next_c],
                board[blank_r][blank_c],
            )
            previous_blank = (blank_r, blank_c)
            blank_r, blank_c = next_r, next_c

        return board

    def blank_pos(self):
        for r in range(self.size):
            for c in range(self.size):
                if self.board[r][c] == 0:
                    return r, c

    def move(self, direction):
        r, c = self.blank_pos()
        dr, dc = {
            "w": (-1, 0),
            "s": (1, 0),
            "a": (0, -1),
            "d": (0, 1),
        }[direction]

        nr, nc = r + dr, c + dc
        if not (0 <= nr < self.size and 0 <= nc < self.size):
            return False

```

```

        self.board[r][c], self.board[nr][nc] = (
            self.board[nr][nc],
            self.board[r][c],
        )
        return True

    def solved(self):
        return (
            sum(self.board, [])
            == list(range(1, self.size * self.size)) + [0]
        )

```

Task 3 testing instructions

- Run python main.py and choose 3, 4, or 5.
- Check each board has the expected dimensions and tile range.
- A legal move increments the move count; an illegal edge move does not.
- Wait during play to see elapsed time advance; after solving, verify the displayed elapsed time is frozen.
- Solve the board and confirm victory is announced once and the game stops accepting moves.
- Enter an invalid board size and confirm the program prompts again.

Task 4 — Success feedback only after an actual move

User prompt

“Now implement Task 4: Fix success feedback so that it occurs only when a tile actually moves.

- Examine how the game currently handles valid and invalid moves.
- Ensure an invalid move leaves the board unchanged.
- Ensure invalid moves do not increment the move counter.
- Ensure invalid moves do not display success messages or trigger victory feedback.
- Display appropriate feedback when a valid tile movement occurs, if the existing game design uses movement feedback.
- Detect victory only after a valid move produces the solved board.
- Ensure the victory message appears only once.
- Prevent any further moves after the puzzle is solved.
- Preserve all existing functionality from Tasks 1, 2, and 3.
- Support 3×3, 4×4, and 5×5.
- Avoid unrelated code changes and unnecessary dependencies.
- Provide the complete updated code for affected files and explain how to test the changes.
- Implement only Task 4.

Task 4 response summary

The game loop was changed to continue immediately after an invalid move, before move counting or victory detection. puzzle.py was also hardened so unsupported directions return False without changing the board. main.py was left unchanged.

game.py after Task 4

```

import time
from puzzle import Puzzle

class SlidingPuzzle:
    def __init__(self, size=4):
        if size not in (3, 4, 5):
            raise ValueError("Puzzle size must be 3, 4, or 5.")

        self.size = size
        self.puzzle = Puzzle(self.size)
        self.moves = 0
        self.started = time.monotonic()
        self.won = False
        self.finished_at = None

    def elapsed_time(self):
        """Return elapsed seconds, frozen when the game is won."""
        end_time = (
            self.finished_at
            if self.finished_at is not None
            else time.monotonic()
        )
        return int(end_time - self.started)

    def display(self):
        print()
        for row in self.puzzle.board:
            print(" ".join(f"{x or ' ':>2}" for x in row))
        print("Moves:", self.moves, " Time:", self.elapsed_time(), "s")

    def _finish(self):
        # Make victory feedback idempotent: announce it only once.
        if not self.won:
            self.won = True
            self.finished_at = time.monotonic()
            self.display()
            print("Congratulations! You solved the puzzle!")

    def run(self):
        # A completed game cannot be resumed or announce victory again.
        if self.won:
            return

        print(
            f"Sliding Puzzle ({self.size}x{self.size}) - "
            "W/A/S/D moves the tile into the blank. Q quits."
        )

        # Preserve behavior for a board already solved before input.
        if self.puzzle.solved():
            self._finish()
            return

        while not self.won:
            self.display()
            key = input("> ").strip().lower()

            if key == "q":
                return
            if key not in {"w", "a", "s", "d"}:
                print("Use W/A/S/D.")
                continue

```

```
# A move is successful only when the board actually changes.
moved = self.puzzle.move(key)

if not moved:
    print("That move is not possible.")
    continue

# Only successful moves reach this point.
self.moves += 1

# Check for victory only after a successful movement.
if self.puzzle.solved():
    self._finish()
    return
```

puzzle.py after Task 4

```

import random

class Puzzle:
    def __init__(self, size=4):
        self.size = size
        self.board = self.make_board()

    def make_board(self):
        # Start from solved order so every generated board is solvable.
        tiles = list(range(1, self.size * self.size)) + [0]
        board = [
            tiles[r * self.size:(r + 1) * self.size]
            for r in range(self.size)
        ]

        # Scramble using legal moves so the board remains solvable.
        blank_r, blank_c = self.size - 1, self.size - 1
        previous_blank = None
        scramble_moves = max(100, self.size * self.size * 20)

        for _ in range(scramble_moves):
            neighbors = []
            for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
                nr, nc = blank_r + dr, blank_c + dc
                if 0 <= nr < self.size and 0 <= nc < self.size:
                    if (nr, nc) != previous_blank:
                        neighbors.append((nr, nc))

            if not neighbors and previous_blank is not None:
                neighbors = [previous_blank]
            if not neighbors:
                break

            next_r, next_c = random.choice(neighbors)
            board[blank_r][blank_c], board[next_r][next_c] = (
                board[next_r][next_c],
                board[blank_r][blank_c],
            )
            previous_blank = (blank_r, blank_c)
            blank_r, blank_c = next_r, next_c

        return board

    def blank_pos(self):
        for r in range(self.size):
            for c in range(self.size):
                if self.board[r][c] == 0:
                    return r, c

    def move(self, direction):
        # Reject unsupported directions without modifying the board.
        directions = {
            "w": (-1, 0),
            "s": (1, 0),
            "a": (0, -1),
            "d": (0, 1),
        }
        if direction not in directions:
            return False

        r, c = self.blank_pos()
        dr, dc = directions[direction]
        nr, nc = r + dr, c + dc

```

```

        # An out-of-bounds move leaves the board unchanged.
        if not (0 <= nr < self.size and 0 <= nc < self.size):
            return False

        self.board[r][c], self.board[nr][nc] = (
            self.board[nr][nc],
            self.board[r][c],
        )
        return True

    def solved(self):
        return (
            sum(self.board, [])
            == list(range(1, self.size * self.size)) + [0]
        )

```

main.py after Task 4

```

from game import SlidingPuzzle

def choose_size():
    """Ask for a supported board size before starting the game."""
    while True:
        choice = input("Choose puzzle size (3, 4, or 5): ").strip()
        if choice in {"3", "4", "5"}:
            return int(choice)
        print("Please enter 3, 4, or 5.")

if __name__ == "__main__":
    size = choose_size()
    SlidingPuzzle(size).run()

```

Task 4 testing instructions

- Run python main.py and test sizes 3, 4, and 5.
- Attempt an edge move that cannot be made; confirm the board is unchanged and the move count does not increase.
- Enter an unsupported key; confirm only the input guidance appears.
- Make a legal move; confirm the board changes and the move count increases by one.
- Prepare a board one legal move away from solved and make that move; confirm victory appears once and the loop exits immediately.
- After winning, call run() again or try gameplay input; the completed game should not resume.
- Confirm the timer advances during play and freezes after victory.

Artifacts referenced in the conversation

- sliding_puzzle_task2_updated.zip — Task 2 project archive.
- sliding_puzzle_task3_updated.zip — Task 3 project archive.
- sliding_puzzle_task4_updated.zip — Task 4 project archive.
- The individual main.py, game.py, and puzzle.py files were also linked in the chat.

End of compiled conversation record.